

The Alberta Buck - Rebalancing the BuckBasket (v1.0)

Waiting for the Turn: an Amortized On-Chain Rebalance Director

Perry Kundert

2026-07-08



The BuckBasket (see *The BuckBasket*) stays roughly balanced at no cost. Redemptions draw from the overweight pools. Deposits and treasury sweeps fill the underweight ones. The world's arbitrageurs do the rest.

What this leaves unused is *timing*. Basket constituents do not move at random. They follow the M2 money supply with lags we can measure, and each commodity has its own lag. A constituent's share of basket value therefore makes long, slow excursions away from its target weight, and comes home when the slower constituents catch up.

This article does four things.

First, it measures those lags and builds a rebalancing policy from them. Scale each constituent's effort by two quantities: how far it sits from its target weight, and how hard its delayed moving average turns back toward that target. Wait while the excursion still grows. Trade when it levels off.

Second, it tests the policy in simulation. The gated policies collect 75-90% of the continuous-rebalancing premium, at a quarter to a half of the turnover, with far better per-trade timing.

Third, it changes coordinates. One window on one share-vs-target deviation collects one timescale only, and it carries the numeraire's noise. *Differential mode* removes both faults: pairwise log ratios, which cancel the common mode exactly, across a filter bank of seven scales. It collects more than continuous rebalancing does, and it is the engine we deploy.

Fourth, and most important, it encodes the policy as an EVM state machine that *amortizes its own maintenance*. Every deposit, redemption or keeper trigger does a small, bounded part of the work. A closed-form catch-up makes a poke after a quiet week cost the same as a poke after a busy hour. The machine then advises the basket through two $O(1)$ reads: where a deposit should go, and which pool best satisfies a redemption.

1 The premise: money moves first, commodities follow

New money does not reach all prices at once. This is the Cantillon effect. Monetary assets reprice in weeks, industrial commodities in months, and wages and food over a year or more.

For the BUCK basket we can measure it. Correlate each constituent's year-over-year growth against M2SL, over a post-1995 window, with lags swept from 0 to 36 months (see `alberta_buck/sim/quotes/fetch` and `rebalance_policy.py --fit-lags`):

constituent	peak lag (months)	peak correlation	tier
cbBTC	0	0.08 (weak)	monetary (fast)
PAXG	censored @36	0.10 (weak)	monetary (prior)
CNST	9-10	0.48-0.52	industrial (mid)
NRGC	13	0.33-0.39	industrial (mid)
LABR	15-16	0.32-0.38	sticky (slow)
FOOD	23	0.21-0.26	sticky (slow)

The order matches the intuition: monetary assets first, then energy and construction, then labour and food. Gold fits poorly, because it trades on real rates and on crisis as much as on M2, so it gets a fast "monetary asset" prior instead of a fitted lag.

The consequence for the basket is this. When M2 surges, the fast constituents overshoot their target share and stay overweight until the slow ones catch up. The lag spread sets how long that takes: roughly a year.

These excursions are the basket's harvest. A rebalancer that reacts to the instantaneous deviation starts to sell the riser immediately, and keeps selling all the way up. It pays fees and bleeds against the trend. A rebalancer that waits for the excursion to level off makes fewer and larger trades, at the maximum mispricing.

2 The policy: quench while diverging, pour on the turn

Per constituent i , at a fixed sampling cadence (one epoch $\sim$ one day):

$\delta_i = w_i/w_i^* - 1$	– raw share deviation, right now;
$m_i = \text{EMA}_{X_i}(\delta_i)$	– the delayed view (X_i -day window);
$v_i = \Delta m_i$	– its velocity: receding / level / gaining;
$\dot{d}_i = \text{EMA}_{7d}(\Delta \delta_i)$	– the observed closure rate.

Each rule below was a *measured failure* in the Python model before it was a rule (`alberta_buck/sim/rebalance_policy.py` make `nix-sim-policy`):

1. **The raw deviation gives direction and size. The moving average gives timing only.** A lagging average can still read "overweight" after the raw share has already crossed back through its target. It must not trade then. Require `sign(m) == sign(delta)`.
2. **The velocity of the moving average classifies the regime.** Still receding, so the deviation grows: do nothing, because the move is not finished. Level, so it turns: start, but slowly. Gaining, so the gap closes: complete the rebalancing.

3. **Size by rate-matching.** Trade at ρ times the closure rate that the 1-week EMA observes: the rate at which we would lose half the distance to balance. A prior of "half the gap in one window" floors it, and that prior applies alone while the regime is level:

$$\text{effort}_i = \min(\rho \hat{c}_i w_i^*, \text{cap}), \quad \hat{c}_i = \max\left(-\text{sign}(\delta_i) \dot{d}_i, \frac{\ln 2}{2X_i} |\delta_i|\right).$$

This calibrates itself: there is no hand-set gain. It also limits itself, because our own flow feeds the rate we observe, so a larger ρ saturates instead of overshooting. Effort peaks just after the turn, when closure accelerates and the deviation is still near its maximum, then falls away as the gap closes.

4. **A leash bounds the wait.** A secular trend never levels off (BTC, 2020-25), so without a bound the deviation would run away. Past

delta > 30% the policy trades at the cap whatever the gate says, and

releases at 25%. The gate times the trades *inside* the leash. The leash enforces the mandate.

5. **Proceeds recycle, but never wash.** Sell-side flow returns to the underweight pools through `sweepTreasury / investFromBucks`. It must never return to a constituent the policy sold in the same round: a stale-signal sell against a raw-underweight rebuy only donates the AMM fee.

The mechanism, live on the 2020-25 history:

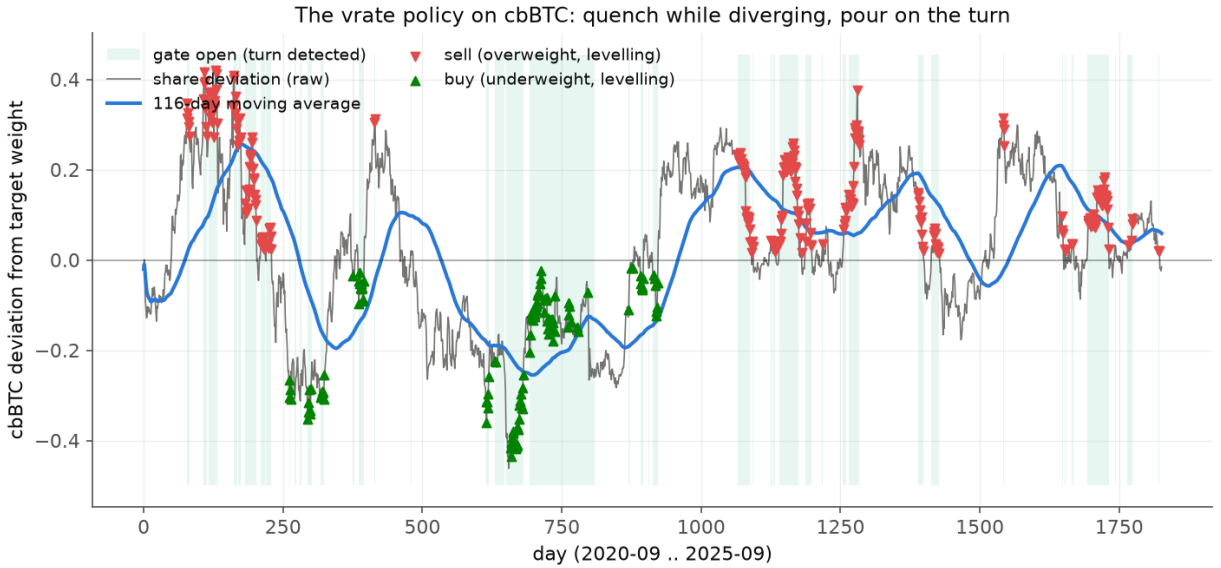


Figure 1: The gated policy on cbBTC's share of the basket, 2020-25. While the excursion is still building (the raw deviation pulling away from its moving average) nothing trades; sells (red) cluster where overweight excursions level off and turn, buys (green) where underweight ones do. Shaded bands mark the gate open.

Measured over 20-year synthetic paths with the fitted lags (cost 30bp/leg, cap 50bp NAV/day), against never rebalancing ("hold") and continuous proportional rebalancing ("prop", what spot-based routing approximates):

policy	premium vs hold	turnover	mean dev at trade	capture90
prop	+423 bp/yr	0.82 NAV/yr	5.3%	+219 bp
band (5%)	+411 bp/yr	1.42 NAV/yr	4.4%	+138 bp
factor (accel)	+381 bp/yr	0.44 NAV/yr	8.8%	+341 bp
vrate (rho=3)	+322 bp/yr	0.25 NAV/yr	12.6%	+514 bp

Each alternative loses to the gated policies on its own terms:

Against hold. Doing nothing gives up the whole rebalancing premium, and it also abandons the mandate: the weights drift wherever prices take them. In the reverting regime every active policy compounds steadily ahead of hold:

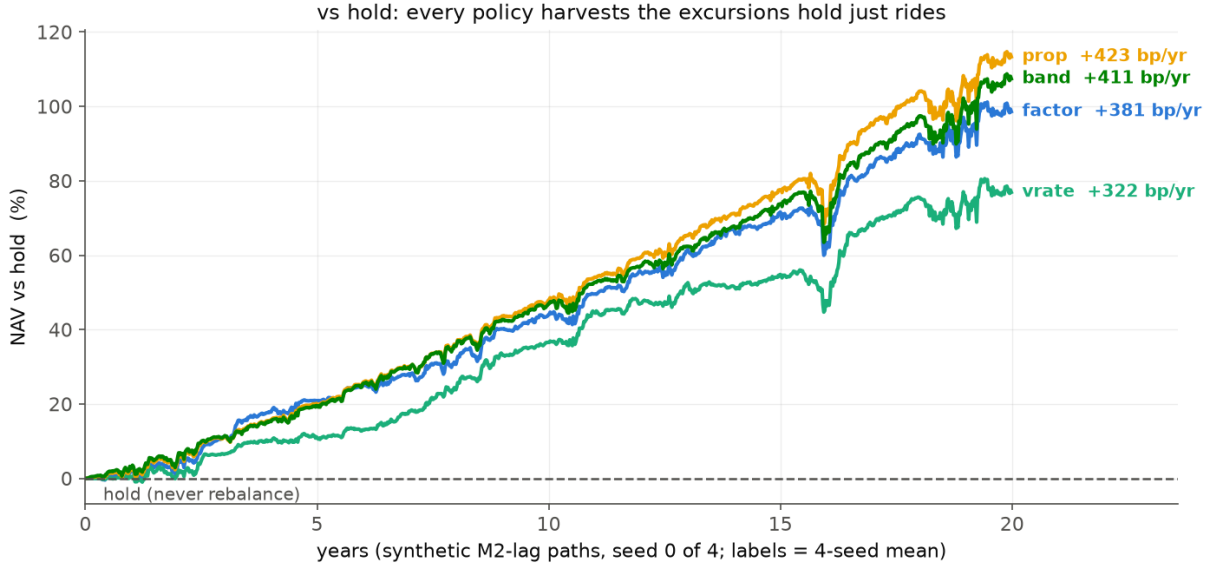


Figure 2: Twenty synthetic years, NAV relative to buy-and-hold (hold = the dashed zero line; line labels give the multi-seed mean premium). The premium is real and persistent – the only question is what it costs to collect.

Against prop, continuous proportional rebalancing, which is what spot-based routing approximates. Prop collects the largest gross premium and pays for it everywhere: the most turnover (0.82x NAV/yr), the shallowest trades (5.3% mean deviation), and the worst behaviour in a secular trend, where it sells a riser continuously all the way up. The 2020-25 replay is the stress case, because BTC ran x9 and reversion never came. There the gated policies lose 90-130bp/yr *less* against the trend, on a half to a third of prop’s turnover, and still hold the weights bounded:

Against band, the 5% threshold rebalancer. Band waits, but for the wrong signal: a fixed deviation level, not the turn of the excursion. It engages as soon as the band is crossed, which is early in a developing excursion. So it trades shallow (4.4% mean deviation), churns more than any other policy (1.42x NAV/yr, from oscillation at the band edge), and times its trades worst. The gated policies wait for the turn itself, and beat band on every per-trade measure:

In short: the gated policies collect 75-90% of prop’s premium, at 30-55% of its turnover, at roughly twice the mispricing depth, and with the least trend-bleed. Full tables are in `BASKET-REBALANCE.md`.

Every policy in that table reads one signal: a constituent’s share of basket value against its target, on one window. The next section replaces that signal, and beats all four. It is also the engine the deployed contract runs. So read the table above as the argument for *gating*, not as the

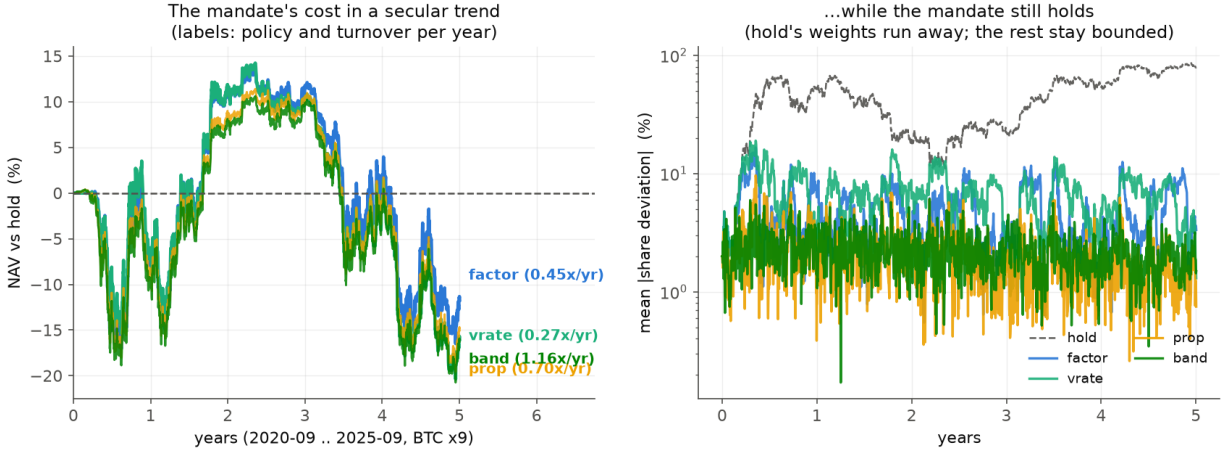


Figure 3: The mandate's cost under a secular trend (2020-25, BTC x9). Left: NAV against buy-and-hold – every constant-mix policy must sell the riser, but the gated policies (factor, vrata) bleed the least, on the least turnover. Right: buy-and-hold's mean absolute deviation runs off toward 80% – it has stopped being the declared basket – while every active policy keeps weights bounded.

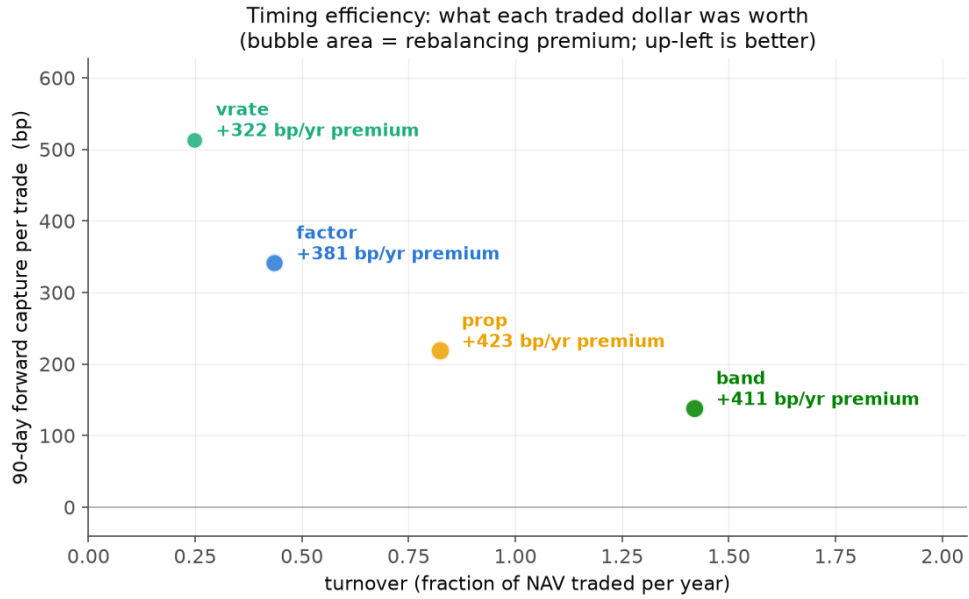


Figure 4: Timing efficiency on the synthetic paths: 90-day forward capture per traded dollar vs turnover (bubble area = premium vs hold). Band is dominated – more churn and worse timing than either gated policy; vrata captures +514bp per traded dollar at 0.25x NAV/yr of turnover.

menu of what ships.

3 Differential mode: the basket as a polyphase machine

The policy above has two weaknesses. Both come from the *coordinate system*, not from the gating.

First, **one window collects one timescale**. Windows fitted to the M2 lag spread catch the year-long excursions, and concede every mid-size swing to whoever rebalances continuously.

Second, **share of basket value is not a clean observable**. We measure a constituent’s share from pool BUCK reserves, so it carries the numeraire’s own noise. When BUCK moves against the dollar, every share moves, and the signal cannot tell that from a commodity that has really become rich.

The reconception (2026-07-11) is to change coordinates. In a three-phase machine you do not reason about the three line voltages. You split them into a **common mode**, the zero sequence, which carries no torque, and the **differential modes**, which carry all of it. The basket has the same structure:

The profit lives in the differentials between commodity legs. The common mode – BUCK’s own valuation against the numeraire – is the K-controller’s job, and it **cancels exactly** in cross-commodity log price ratios.

So the **pairs** engine works pairwise, and in logarithms. For legs i, j the imbalance is

$$d_{ij} = \ln \frac{1 + \delta_i}{1 + \delta_j},$$

This is numeraire-free by construction: any factor common to both legs subtracts out. A Uniswap *tick is already a log price*, so the engine reads ticks straight from `slot0`, and never needs `ln()` on-chain.

3.1 A filter bank instead of a window

Each leg carries $K = 7$ EMAs of its normalized tick, on geometrically spaced windows: 5, 10, 20, 40, 80, 160 and 320 days. The EMA is linear, so *every pair’s moving average, velocity and curvature at every scale is the difference of two legs’ ladders*. The full $N(N - 1)/2$ differential graph therefore falls out of $O(NK)$ state, with no pairwise storage at all.

The difference of two adjacent EMAs is a band-pass filter, so the ladder is an octave filter bank. The short windows see mid-size swings. The long ones see the M2 excursions. The same gating logic applies at each scale, and each scale decides on its own.

3.2 The quorum turn detector

Each scale casts one vote per pair. A scale votes only when both single-scale rules hold at that scale: its moving-average gap agrees in sign with the raw pairwise imbalance, *and* its curvature points back toward equilibrium. The pair opens when the votes reach a quorum, 4 of 7 by default. No scale can fire alone, which is what makes a fast window safe to include.

Effort is $\kappa |d_{ij}| (\text{votes}/K)$, capped, under a pairwise leash with hysteresis at 30% and 25%. Trades are **matched pairs**: sell the rich leg, buy the poor leg, equal value. They are therefore self-financing, they leave no cash residue, and they are wash-proof by construction, because the engine cannot sell a constituent and rebuy it in the same round. The single-scale policy needed an explicit no-wash rule to prevent that. A progressive warmup lets the short windows vote from about day 20, so the engine is not silent while a 116-day window fills.

3.3 What it measures

Synthetic 20-year paths, 5 seeds, quorum 4, premium vs hold in bp/yr:

cost/leg	prop	band	factor	vrata	pairs
30bp	+430	+417	+393	+340	+496
100bp	+365	+308	+357	+313	+422
250bp	+228	+89	+283	+271	+262

pairs beats even *prop's gross harvest*, for two reasons: multi-scale coverage recovers the mid-size swings, and pair-matched execution collects more per unit of imbalance. It dominates every policy through roughly 200bp/leg of cost. Only at extreme costs does **factor** retake the lead.

The quorum trades premium against cost: q3 gives the most premium (+508, at 1.39 turnover), q5 the least turnover (0.65, at +460), and **q4 is the knee**.

The vote criterion is the other knob, and it decides the engine's character (quorum 4):

vote	criterion	synth premium	turnover	capture90	hist CAGR
curv	divergence <i>decelerating</i> – early, at the top	+496	0.96	+201 bp	17.9%
vel	gap <i>already closing</i> – the confirmed turn	+421	0.60	+270 bp	18.4%

curv buys about 75bp/yr more gross harvest, for 60% more turnover and worse trend bleed. The two cross at roughly 235bp/leg of cost. **vel** trades later and less, with better per-trade capture and trend robustness equal to *prop's*. **vel** is **the deployment default**, and it is the criterion `PairsRebalanceDirector.sol` implements.

3.4 Which engine, and when

The whole study comes down to a regime split:

- **pairs is the reverting-regime pick** – the regime the *BuckBasket* is built for, and the one the physics argument in *The BuckBasket* says is the default behaviour of the physical economy.
- **factor remains the trend-regime pick** – best historical CAGR (19.3%) through the 2020-25 replay, where BTC ran x9 and reversion never arrived.

Between the two engines that *exist on-chain* the choice is not close. **pairs** beats **vrata** on the premium and on the risk measure. **vrata** concentrates its trades where the observed closure is fastest. In a trend that means selling into sharp V-shaped pullbacks which then resume, and its historical capture90 is the worst of any policy studied: -676bp, against +270bp for **pairs** / **vel**. In the reverting regime the same concentration is exactly right, at +514bp, the best of any policy. That is what makes **vrata** the wrong default: of all the policies, its result depends most on the regime being the one you hoped for.

4 Encoding it on-chain: work that amortizes across whoever shows up

The policy needs a clock, because moving averages advance with time. The EVM has no scheduler, and nobody pays to run one. The established answer is the pattern behind Compound's `accrueInterest`, MakerDAO's per-collateral `drip` and Uniswap's oracle observations:

Lazy time-indexed accrual. State is advanced on first touch, as a closed-form function of elapsed time – never per block.

Three refinements make it fit a *multi-pool signal aggregator*:

4.1 Closed-form EMA catch-up

Under sample-and-hold (the deviation observed now stands in for the quiet gap), n missed epochs of EMA recursion collapse exactly, with $q = 1 - \beta$ (and q_7 the one-week EMA's decay):

$$\begin{aligned} m' &= \delta + (m - \delta) q^n && \text{– the average;} \\ v' &= q^n v - (1 - q)^2 n q^{n-1} (m - \delta) && \text{– its velocity;} \\ d' &= q_7^{n-1} ((\delta - \delta_{\text{prev}})(1 - q_7) + q_7 d) && \text{– the closure rate;} \end{aligned}$$

Binary exponentiation computes each in $O(\log n)$. One poke after a quiet month costs the same as one poke after a busy hour, *and it lands the signals where n diligent pokes would have left them*. This is not approximately true. It is an invariant that the test suite fuzzes over 256 random gap-and-imbalance combinations, and enforces to fixed-point rounding.

The closed form for the velocity matters. A naive "average dm over the gap" update classifies the regime differently from the diligent recursion. That was a real bug, and the regime test matrix caught it.

4.2 The round-robin work wheel

Take N constituents, a persistent cursor and `poke(maxWork)`. Each call advances at most `maxWork` stale constituents, starting from the cursor. Every basket activation – a deposit, a redeem, a treasury sweep or a bare keeper trigger – carries a small budget of 1 to 2. The measured gas, on 6-token mocks with `via-IR` and `solc 0.8.28`:

operation	gas
poke one constituent, 1-epoch gap	~36,500
poke one constituent, 50-epoch gap	~19,900
poke when everything is fresh	~3,100

Catch-up is *flat in the size of the gap*. The 50-epoch poke is cheaper than the 1-epoch poke only because its storage is already warm. The fresh-epoch guard is almost free. **So the more often the machine is triggered, the less work each agent does.** If nobody arrives for a week, whoever arrives next pays one closed-form update, not a week of them. Work is conserved. Granularity trades per-call gas against signal staleness, and never against total cost.

4.3 Cached observations with running sums

A constituent's deviation is its share of the total, so it needs *everyone's* pool values. Reading N pools per poke would defeat the sharding. So each poke reads only *its own* pool: one `balanceOf` for the BUCK reserve bv_i (a full-range position has pool value $2bv_i$), and one `slot0` for the spot, which gives the price-scaled target share $s_i = a_i P_{0,i}^2 / \text{spot}_i$. That is the same fixed-quantity-index target the pro-rata redemption uses. The poke then folds its reading into cached running sums, by difference:

$$B += bv_i^{\text{new}} - bv_i^{\text{cached}}, \quad S += s_i^{\text{new}} - s_i^{\text{cached}}, \quad \delta_i = \frac{bv_i/B}{s_i/S} - 1.$$

The sums are slightly stale, and a moving-average policy is insensitive to that by construction. Decision data aggregates the same way. Each poke stores its constituent’s signed advisory effort, so the two decision reads are simple scans over N cached `int32`s: `depositHint()` returns the strongest buy-side effort, and `redeemHint()` the strongest sell-side.

4.4 The trust model: hints, not authority

The director holds no funds and executes nothing. Its outputs are *advisory*. On every trade it performs, the basket’s venue re-verifies everything material: the spot-versus-TWAP manipulation guard, the liquidity bounds and the conversion-loss budget.

A hint that is stale, wrong or even adversarial can therefore only re-order flows – send a deposit to the second-best pool, say. It cannot break solvency, and it cannot extract value beyond what the venue’s own guards already bound. That is why the signal state is safe to maintain lazily, through unauthenticated public pokes.

5 The contracts: two engines on a shared chassis

`RebalanceDirectorBase.sol` is the policy-agnostic chassis – epoch clock, `poke(maxWork)` work wheel, metadata sync, cached observations with running sums, binary exponentiation, and the `depositHint` / `redeemHint` / `effortOf` surface – and two signal engines plug into it behind the common `IRedirector` advisory interface:

- `BasketRebalanceDirector.sol` (**vrate**) – the single-window share-vs-target policy derived above.
- `PairsRebalanceDirector.sol` (**pairs**) – differential mode: per-leg 7-window tick-EMA ladders, pair quorum votes on the confirmed turn (**vel**), arithmetic weight-ratio differential, pairwise leash bitmask, and `bestPair()` naming the top matched trade for keepers. Pair efforts are folded into per-leg *net* efforts at poke time, so the chassis’s advisory reads stay $O(N)$ cached – fully API-compatible with **vrate**.

pairs at quorum 4 is what deploys, and what the rebalancing simulation runs. The rest of this section describes **vrate**, because its state layout shows the shared machinery more simply.

`src/basket/BasketRebalanceDirector.sol` (~2.5 slots of hot state per constituent, packed):

```
struct Signal {
    int128 m;           // EMA_X of deviation (1e18)
    int128 vEma;        // EMA_X of per-epoch d(m)
    int128 ddotEma;     // 1-week EMA of per-epoch d(delta)
    int128 prevDelta;
    uint128 bv;         // cached pool BUCK reserve
    uint128 s;          // cached price-scaled target share
    int32 effortBp;     // signed advisory effort (bp of NAV per epoch)
    uint32 lastEpoch;
    uint32 firstEpoch;
    bool leashed;
}
```

The API surface:

- `syncConstituents()` – permissionless; pulls constituent metadata from the basket’s public getters (re-callable: `addBasketToken` renormalizes).
- `poke(maxWork)` / `pokeAll()` – the work wheel. Skips fresh constituents for ~ 2 warm SLOADs; advances stale ones with the closed forms above and the vrate effort rule (regime gate, rate-matched size, leash, warmup).
- `depositHint()` / `redeemHint()` / `effortOf(i)` / `effortsAll()` – the $O(1)/O(N)$ -cached advisory reads. A `type(uint256).max` hint means "no signal; use the default most-underweight routing".
- `setParams(...)` – governance: epoch length, window, ρ , deadband, regime band, leash bounds, per-epoch cap.

How `BuckBasketProRata` consumes it (the integration increments, in order of invasiveness):

1. **Increment 0 – flow routing, no new trades.** `_depositBuck` and `sweepTreasury` pass `director.depositHint()` to `investFromBucks` as the `poolHint`, and fall back to most-underweight on the sentinel. The single-token redeem path reads `redeemHint()`. Each of these activations also calls `director.poke(1)`, the bounded slice that keeps the signals alive. The basket’s ordinary deposit, redeem and sweep traffic then performs well-timed rebalancing *for free*.
2. **Increment 1 – keeper rebalance steps.** A permissionless pair, `rebalanceSource(i)` (withdraw + convert from the strongest sell-side pool, bounded by `effortBp`) and `rebalanceSink()` (invest the pending BUCK into the deposit hint), each one small transaction – the `sweepTreasury` pattern, directed by the signals.
3. **Weighted draws.** `_allocateSellHigh` can tilt its per-pool draw weights by `effortsAll()` rather than spot overweight alone.

6 Verification

Three layers, all runnable from the Makefile:

(The model figures above regenerate via `make nix-sim-policy` then `make nix-sim-plot-article`.)

- **Unit and fuzz tests (Forge)** – `make nix-test-director`. Ten tests cover six things. The amortization invariant, as diligent-versus-lazy twins, plus a 256-run fuzz over random gaps and imbalances. The regime transitions on a scripted excursion: receding waits, level starts, gaining saturates, and sign-agreement blocks the wrong-way trade from a stale moving average. Leash engagement and its hysteresis. Both hints. Wheel budgets and cursor rotation. And the gas figures quoted above.
- **Parallel regime matrix** – `make nix-test-director-regimes`. The same suite re-run across a window x rho grid (defaults: $\{6, 8, 16\}$ epochs x $\{1.5, 3, 6\}$), one forge process per regime, in parallel. This matrix is what exposed the velocity catch-up approximation: windows of 6 classified a post-gap regime differently between the lazy and diligent paths until the closed form replaced the average-velocity shortcut.

- **End-to-end on Anvil** – make `nix-sim-director`. This runs the rebalancing scenario, about 400 agents: arbs, LPs, direct-mint depositors and a whale. A `DirectorKeeperAgent` pokes the deployed director with a 2-constituent budget each tick, and executes its advice as 2-hop Universal Router swaps, sell-hint to BUCK to buy-hint, sized by the advised bp of basket NAV. The output vector records `directorPokes` and `directorTrades` per day, beside the existing basket telemetry.

7 Variants worth investigating

Nothing in this section is measured. These are the directions the machine’s own structure suggests, each with the literature that would supply the theory, ordered by expected return on effort. Two analogies organize them, and both do real work rather than decoration.

7.1 Taking the machine analogy literally

The three-phase intuition that produced differential mode has more in it. A polyphase drive does not stop at "separate the common mode". It applies two specific transforms, and each has an exact basket counterpart:

- The **Clarke transform** (abc to alpha-beta-zero) splits a three-phase vector into a zero sequence, which carries no torque, and two *orthogonal* differential components. The basket’s log-weight vector splits the same way: one common mode, BUCK against the numeraire, which is the K-controller’s business, and $N - 1$ differential modes.
- The **Park transform** (alpha-beta to d-q) then rotates into a frame that turns with the field. Quantities that were sinusoidal become constant, and a plain PI controller is then enough. Field-oriented control uses this to split the current into a torque-producing part and a flux-producing part, and regulates each separately.

The basket’s d-q split is the useful part. One part of an imbalance is *harvestable*, because it reverts: that is torque. The other part is *secular*, a trend that must be tolerated and bounded: that is flux. The leash is a crude form of this distinction today, applied after the fact and with a hand-set bound.

Slip is the concept the engine already rediscovered. Induction torque is proportional to slip, the *relative* rate between field and rotor, and it falls to zero at zero slip however fast both are turning. That is the rate-matching rule exactly: the harvest scales with the rate of relative movement, not with absolute price motion. The machine analogy therefore confirms the policy’s central choice after the fact.

7.2 Taking the frequency analogy literally

The EMA ladder is already a decomposition in time: geometrically spaced windows, with the difference of two adjacent EMAs acting as a band-pass filter. That is an octave filter bank, which is a wavelet transform under another name. The cross-sectional axis is *not* decomposed. The full object is two-dimensional, **mode x timescale**, and the engine does one axis properly and the other by brute force.

7.2.1 Orthogonal modes instead of pairs

The sharpest criticism of the **pairs** engine is that its basis is redundant. Six legs carry **five** independent differential modes, but the engine tracks **fifteen** pairs. The quorum therefore counts overlapping evidence as if it were independent, and the effort sizing counts the same excursion two or three times.

The fix is the Clarke transform, generalized: diagonalize the covariance of the leg log-returns, and work in the eigenbasis. PC1 is the common mode, and here it is *discarded*, because it is the controller's job. PC2 to PCN are orthogonal, so each can be sized on its own without double-counting, and a quorum over modes is a quorum over genuinely independent evidence. This is the standard construction in statistical arbitrage (Avellaneda and Lee, *Statistical Arbitrage in the US Equities Market*, 2010: PCA eigenportfolios with mean-reverting residuals).

The on-chain cost is an $N \times N$ fixed-point rotation of the cached ladders. At $N = 6$ that is small, and the eigenvectors are governance parameters, refitted rarely rather than per poke. The cointegration literature (Johansen) gives a more principled version again: for constituents bound to each other by physics, the object to trade is the *stationary* linear combination, not the raw pairwise imbalance.

7.2.2 Trade to the boundary, not to the target

Under proportional transaction costs, the optimal policy is provably **not** to trade toward the target. It is a **no-trade region**, and one trades only far enough to return to its boundary (Davis and Norman 1990; Shreve and Soner 1994). The current engine sizes effort toward proportion, and relies on the cap and the deadband to stop it overshooting. Make the deadband a genuine no-trade wedge, and size effort to reach the near edge instead of the centre. This is the cheapest change on this list, and the one with the firmest theory behind it.

7.2.3 Bands fitted to the reversion, not set by hand

The deadband is a flat 150bp, and the leash a flat 30%. For a mean-reverting spread, the optimal entry and exit thresholds are known in closed form, as a function of the reversion rate, the volatility and the cost (Bertram 2010; Leung and Li on optimal Ornstein-Uhlenbeck trading). They scale like $\sigma/\sqrt{2\theta}$. The M2-lag study already fits a timescale per constituent, so fitting (θ, σ) per *pair* or per *mode* is the same kind of exercise. It would replace two hand-set constants with measured ones.

7.2.4 Weights are bounded, so bound them properly

"Strict upper and lower bounds" has an exact stochastic counterpart. Weights live on a simplex, and the natural diffusion there is the **Jacobi (Wright-Fisher)** process: its stationary distribution is Beta, and its first-passage behaviour is known. Fit it, and the leash becomes a *quantile* instead of a guess. "This excursion is at the 99th percentile of what this constituent's share does" is a defensible trigger. "30%" is not.

7.2.5 A target-share family with a theorem attached

Constant-mix is one point in a family. Fernholz's stochastic portfolio theory shows two things. A portfolio generated by a concave function of the market weights decomposes into a generating term plus a drift term, and that drift term is *non-negative*. And **diversity-weighted** portfolios ($w_i \propto \mu_i^p$, $0 < p < 1$) beat the market portfolio under a diversity condition – which is exactly the bounded-weights premise the BuckBasket rests on.

The rebalancing premium this article measures *is* Fernholz’s excess growth rate. Expose p as a governance parameter and you get a continuous knob, from equal-weight (maximum premium, maximum turnover) to cap-weight (neither), with a proof behind the interpolation instead of a simulation.

7.2.6 Phase, not just velocity

"Wait for the turn" is a statement about *phase*: trade when an excursion nears the end of its cycle. The engine infers this from the sign of a velocity or a curvature, which is a one-bit reading of a continuous quantity. The analytic-signal construction – a Hilbert transform, approximated cheaply by an EMA quadrature pair – gives instantaneous amplitude and phase directly. Effort could then be a smooth function of how far through its excursion a pair actually is.

Energy and food have real seasonality, so genuine periodicity is worth detecting. The **Goertzel algorithm** computes one DFT bin from two state variables, at $O(1)$ work per sample. It is the one FFT-family primitive cheap enough to run on-chain unmodified. It pays only if a fitted seasonal component turns out to be material, which is a reason to *test* for one.

7.3 What not to do

Two directions sound attractive and are traps. The article’s own data says so.

Do not chase gross premium. `curv` and `quorum 3` each buy more harvest with more turnover, and the result of this whole study is that timing beats volume.

Do not add a non-reverting constituent to sharpen the signal. Every gear in this machine assumes reversion, so feeding it something built not to revert cannot improve it. *The BuckBasket* makes the argument in full, and the 2020-25 replay is the measurement.

8 Limitations and next increments

- The lazy catch-up is exact under sample-and-hold, but a real gap has a price path. More pokes give more fidelity at the same amortized price, and the basket’s own activity supplies them exactly when flows are high, which is when fidelity matters most.
- The director reads whole-pool BUCK reserves, while the venue nets out treasury liquidity. The bias is small, and it does not affect the *order* of the hints.
- Windows are governance parameters, seeded from the M2-lag analysis. The model says the premium is flat from 30 to 180 days, so mis-tuning them costs little.
- The basket-side consumption is the next contract pass: the increment-0 wiring in `BuckBasketProRata`, and the `rebalanceSource/Sink` pair. The director itself, its tests and the sim keeper are complete.
- Keeper bounty economics are not modelled yet. A public network needs them: a few bp of each productive step, paid from the treasury.
- **The trend-regime pick has no on-chain engine.** `factor` wins the historical replay, but it exists only as a Python policy: the two Solidity engines are `vrata` and `pairs`. So either we accept the regime split as a deliberate bet on reversion – which the basket’s whole thesis already is – or `factor` needs an engine, so that governance can switch.

- The **pairs** basis is redundant (fifteen pairs over five independent modes at $N = 6$); see *Variants worth investigating*.